Resume-to-Opportunities Engine

Detailed Project Specification and Development Blueprint

1. Project Vision

Build an intelligent platform that allows users to upload their resume once and automatically discover highly relevant jobs and internships. The platform continuously searches opportunities, scores their relevance, identifies missing skills, and notifies users about new matches.

2. Problem Statement

Students and early-career professionals spend significant time manually searching job portals, filtering irrelevant postings, and customizing applications. Existing job boards are fragmented and rarely provide personalized skill-gap insights. This project aims to centralize and automate the process.

3. Goals and Objectives

Primary goals include: automated resume analysis, personalized job recommendations, daily opportunity discovery, skill-gap analysis, notifications, and eventually tailored resume generation.

4. Target Users

Students seeking internships, fresh graduates, career switchers, and junior software engineers looking for opportunities that match their existing skills.

5. Core Features

Features include: resume upload, skill extraction, profile creation, automated job searching, relevance scoring, explanations for recommendations, skill-gap analysis, notifications via email or Telegram, bookmarks, and eventually AI-assisted resume tailoring.

6. System Architecture

Frontend: Next.js. Hosting: Vercel. Database and authentication: Supabase. Resume parsing: PyMuPDF and spaCy. Job acquisition: public job APIs and selected scrapers. Notifications: Telegram Bot API and email services. Scheduling: GitHub Actions or Vercel Cron Jobs.

7. Functional Modules

Modules include authentication, resume ingestion, text extraction, NLP pipeline, job aggregation engine, matching engine, recommendation system, notifications, and analytics.

8. Resume Processing Pipeline

User uploads PDF → Extract text using PyMuPDF → Normalize text → Detect sections such as education and experience → Extract skills and technologies → Generate user profile.

9. Skill Extraction Strategy

Initially use rule-based extraction with curated dictionaries of programming languages, frameworks, cloud technologies, databases, and tools. Later versions can introduce local LLMs for semantic extraction.

10. Job Search Engine

The system periodically queries APIs and scrapers, stores jobs in a database, deduplicates results, normalizes fields, and indexes jobs for efficient retrieval.

11. Matching Algorithm

Calculate scores based on exact skill matches, related technologies, role similarity, location preferences, internship requirements, and recency. The engine should also provide explanations describing why a particular opportunity is recommended.

12. Skill-Gap Analysis

Compare user skills against required job skills and identify missing technologies. Provide learning recommendations and track progress over time.

13. Notification System

Scheduled tasks search for new opportunities and send personalized alerts. Messages include job title, company, location, match score, and missing skills.

14. Database Design

Suggested tables: users, resumes, extracted_skills, jobs, applications, notifications, bookmarks, and search_history.

15. Non-Functional Requirements

The system should be responsive, scalable, secure, fault-tolerant, and inexpensive to operate. Initial deployment target is essentially zero monthly cost.

16. Development Phases

Phase 1: Resume upload and parsing. Phase 2: Job aggregation and scoring. Phase 3: Notifications and saved searches. Phase 4: Tailored resumes and advanced AI features.

17. Future Enhancements

Semantic search, company recommendations, interview preparation, portfolio analysis, learning roadmaps, analytics dashboards, and mobile applications.

18. Educational Value

This project demonstrates full-stack development, NLP, search algorithms, data engineering, scheduling, APIs, automation, system design, and product thinking.

Suggested Folder Structure

frontend/, backend/, services/resume-parser/, services/job-search/, services/matching-engine/, database/, cron-jobs/, notifications/, docs/, and tests/.

Suggested Milestones

Week 1: Project setup and resume parsing. Week 2: Skill extraction and database design. Week 3: Job search integration. Week 4: Matching engine and recommendations. Week 5: Notifications and deployment. Week 6: Refinement and documentation.